

UNIVERSIDADE DO VALE DO ITAJAÍ – UNIVALI
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Marcelo Mafra de Moura
marcelo.8536600@edu.univali.br

Bernardo Ferraz de Campos
bernardo.8583501@edu.univali.br

SERVIÇOS WEB PARA PAINÉIS GRÁFICOS DE GESTÃO IMOBILIÁRIA

Entrega 1 – Hands on Work VII

Relatório apresentado à disciplina de Hands on Work VII, do curso de Análise e Desenvolvimento de Sistemas da UNIVALI, referente à modelagem do banco de dados e ao desenvolvimento dos serviços web associados ao projeto de extensão Núcleo de Apoio ao Migrante (NAM).

Itajaí (SC)
2026

SUMÁRIO

INTRODUÇÃO	1
MODELAGEM DO BANCO DE DADOS	1
CARGA DE DADOS	2
CONSULTA COM JUNÇÃO ENTRE AS TABELAS	4
CÓDIGO DE ACESSO AO BANCO DE DADOS	4
database-connection.ts	5
pagamento-repository.ts	6
consultar-pagamentos.ts	7
Resultado da execução	8
ESPECIFICAÇÃO OPENAPI DOS SERVIÇOS DA PARTE 2	8
CONSIDERAÇÕES FINAIS	11
REFERÊNCIAS	12

INTRODUÇÃO

Este relatório documenta a primeira entrega do trabalho da disciplina de Hands on Work VII, do curso de Análise e Desenvolvimento de Sistemas da UNIVALI. A proposta da disciplina é construir serviços web de backend que consigam alimentar painéis gráficos de uma gestora imobiliária, sem se preocupar com a construção das telas em si.

O trabalho está vinculado ao projeto de extensão Núcleo de Apoio ao Migrante (NAM), coordenado pelo professor Rafael Padilha dos Santos, cujo objetivo é apoiar migrantes em situação de vulnerabilidade em Santa Catarina, inclusive na busca por moradia e trabalho. Os dados de pagamentos de imóveis processados aqui servem de base para decisões futuras sobre viabilidade de moradia nessas regiões.

Nesta primeira etapa, o grupo modelou o banco de dados relacional, carregou os registros de exemplo, escreveu a consulta que junta as tabelas e o código em TypeScript que a executa, além de especificar em OpenAPI os três serviços REST que serão implementados na Parte 2. Um ponto central do enunciado é que o SGBD não pode filtrar nem agrupar dados: toda essa responsabilidade fica com a aplicação, usando técnicas de programação funcional sobre os dados carregados por inteiro em memória.

O banco utilizado foi o MySQL e o backend foi escrito em TypeScript, rodando sobre Node.js, com a camada de acesso a dados organizada de forma orientada a objetos.

MODELAGEM DO BANCO DE DADOS

O banco ficou dividido em três tabelas ligadas por chave estrangeira: `tipo_imovel`, `imovel` e `pagamento`. Um imóvel pertence a exatamente um tipo (apartamento, casa, terreno ou sala comercial) e pode ter vários pagamentos ao longo do tempo, o que representa o histórico de aluguel de cada um deles.

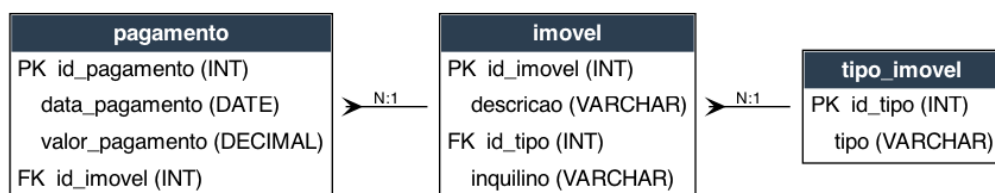


Figura 1 – Diagrama entidade-relacionamento do banco de dados

A criação das tabelas ficou assim:

```
CREATE TABLE tipo_imovel (
    id_tipo INT NOT NULL AUTO_INCREMENT,
    tipo VARCHAR(30) NOT NULL,
    PRIMARY KEY (id_tipo)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE imovel (
    id_imovel INT NOT NULL AUTO_INCREMENT,
    descricao VARCHAR(400) NOT NULL,
    id_tipo INT NOT NULL,
    inquilino VARCHAR(70) DEFAULT NULL,
    PRIMARY KEY (id_imovel),
    KEY id_tipo (id_tipo),
    CONSTRAINT imovel_ibfk_1 FOREIGN KEY (id_tipo) REFERENCES tipo_imovel (id_tipo)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE pagamento (
    id_pagamento INT NOT NULL AUTO_INCREMENT,
    data_pagamento DATE NOT NULL,
    valor_pagamento DECIMAL(12,2) NOT NULL,
    id_imovel INT NOT NULL,
    PRIMARY KEY (id_pagamento),
    KEY id_imovel (id_imovel),
    CONSTRAINT pagamento_ibfk_1 FOREIGN KEY (id_imovel) REFERENCES imovel (id_imovel)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

O script completo, já com a carga de dados, está em `db/schema.sql` no repositório do trabalho.

CARGA DE DADOS

O enunciado pede pelo menos 30 pagamentos espalhados em 5 meses diferentes, e pelo menos 8 imóveis cobrindo 3 tipos distintos, todos com pagamento associado. Para não ficar no limite,

o grupo inseriu 36 pagamentos entre agosto e dezembro de 2025 (5 meses), distribuídos entre 8 imóveis de 4 tipos (apartamento, casa, terreno e sala comercial).

```
SELECT COUNT(*) AS total_pagamentos,
       COUNT(DISTINCT DATE_FORMAT(data_pagamento, '%m/%Y')) AS meses_distintos
FROM pagamento;
```

total_pagamentos	meses_distintos
36	5

```
SELECT COUNT(*) AS total_imoveis,
       COUNT(DISTINCT id_tipo) AS total_tipos_distintos
FROM imovel;
```

total_imoveis	total_tipos_distintos
8	4

O enunciado também exige que todo imóvel cadastrado tenha ao menos um pagamento associado. Para confirmar isso, foi feito um LEFT JOIN de imovel com pagamento, contando quantos pagamentos cada imóvel possui:

```
SELECT i.id_imovel, COUNT(p.id_pagamento) AS qtd_pagamentos
FROM imovel i
LEFT JOIN pagamento p ON p.id_imovel = i.id_imovel
GROUP BY i.id_imovel
ORDER BY i.id_imovel;
```

id_imovel	qtd_pagamentos
1	5
2	5
3	5
4	5
5	5
6	5

id_imovel	qtd_pagamentos
7	3
8	3

Nenhum imóvel ficou com zero pagamentos, o que confirma que a exigência foi atendida.

CONSULTA COM JUNÇÃO ENTRE AS TABELAS

A consulta abaixo junta as três tabelas e devolve a estrutura pedida no enunciado (id_venda, data_do_pagamento, valor_do_pagamento, codigo_imovel, descricao_imovel, tipo_imovel). Ela não usa WHERE nem GROUP BY, exatamente porque o enunciado pede que o filtro e o agrupamento aconteçam na aplicação, e não no banco.

```
SELECT
  p.id_pagamento    AS id_venda,
  p.data_pagamento  AS data_do_pagamento,
  p.valor_pagamento AS valor_do_pagamento,
  i.id_imovel        AS codigo_imovel,
  i.descricao        AS descricao_imovel,
  t.tipo             AS tipo_imovel
FROM pagamento p
JOIN imovel i ON i.id_imovel = p.id_imovel
JOIN tipo_imovel t ON t.id_tipo = i.id_tipo
ORDER BY p.data_pagamento;
```

Esse arquivo está salvo separadamente em db/consulta-join.sql, para ser reaproveitado pelo código TypeScript apresentado na próxima seção.

CÓDIGO DE ACESSO AO BANCO DE DADOS

Para atender ao pedido de usar orientação a objetos, o acesso ao banco foi organizado em duas classes: DatabaseConnection, que só cuida do pool de conexões com o MySQL, e PagamentoRepository, que sabe executar a consulta da seção anterior. Um terceiro arquivo, consultar-pagamentos.ts, monta essas duas peças e imprime o resultado. O projeto todo

foi escrito em TypeScript, rodando direto via tsx, sem precisar de um passo de build separado.

database-connection.ts

```
import mysql, { type Pool, type RowDataPacket } from 'mysql2/promise';

interface DatabaseConnectionConfig {
  host: string;
  port: number;
  user: string;
  password: string;
  database: string;
}

export class DatabaseConnection {
  private readonly pool: Pool;

  constructor({ host, port, user, password, database }: DatabaseConnectionConfig) {
    this.pool = mysql.createPool({
      host,
      port,
      user,
      password,
      database,
      waitForConnections: true,
      connectionLimit: 5,
    });
  }

  async query<T extends RowDataPacket[]>(sql: string, params: any[] = []): Promise<T> {
    const [rows] = await this.pool.execute<T>(sql, params);
    return rows;
  }
}
```

```

    async close(): Promise<void> {
        await this.pool.end();
    }
}

```

pagamento-repository.ts

```

import { readFileSync } from 'node:fs';
import { fileURLToPath } from 'node:url';
import path from 'node:path';
import type { RowDataPacket } from 'mysql2/promise';
import { DatabaseConnection } from '../config/database-connection.js';

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const CONSULTA_JOIN_SQL = readFileSync(
    path.join(__dirname, '../db/consulta-join.sql'),
    'utf8'
);

export interface PagamentoCompleto extends RowDataPacket {
    id_venda: number;
    data_do_pagamento: string;
    valor_do_pagamento: string;
    codigo_imovel: number;
    descricao_imovel: string;
    tipo_imovel: string;
}

export class PagamentoRepository {
    constructor(private readonly databaseConnection: DatabaseConnection) {}

    async buscarSerieHistoricaCompleta(): Promise<PagamentoCompleto[]> {

```

```

        return this.databaseConnection.query<PagamentoCompleto[]>(CONSULTA_JOIN_SQL);
    }
}

```

consultar-pagamentos.ts

```

import 'dotenv/config';
import { DatabaseConnection } from '../config/database-connection.js';
import { PagamentoRepository } from '../repositories/pagamento-repository.js';

async function main(): Promise<void> {
    const databaseConnection = new DatabaseConnection({
        host: process.env.DB_HOST as string,
        port: Number(process.env.DB_PORT),
        user: process.env.DB_USER as string,
        password: process.env.DB_PASSWORD ?? '',
        database: process.env.DB_NAME as string,
    });

    const pagamentoRepository = new PagamentoRepository(databaseConnection);

    try {
        const serieHistorica = await pagamentoRepository.buscarSerieHistoricaCompleta()
        console.log(`Total de registros carregados em memoria: ${serieHistorica.length}`)
        console.table(serieHistorica);
    } finally {
        await databaseConnection.close();
    }
}

main().catch((error) => {
    console.error('Falha ao consultar pagamentos:', error);
    process.exitCode = 1;
})

```

```
});
```

Resultado da execução

Rodando `npm start`, os 36 registros aparecem carregados de uma vez em memória, sem nenhum filtro ou agrupamento feito pelo banco. Uma amostra do resultado:

id_venda	data_pagto	valor_pagto	cod_imovel	tipo_imovel
1	2025-08-05	5000.00	1	Apartamento
2	2025-08-10	4500.00	2	Apartamento
3	2025-08-15	7000.00	3	Casa
4	2025-08-20	6000.00	4	Casa
5	2025-08-25	3000.00	5	Terreno
...	...	...	...	...
36	2025-12-30	3800.00	8	Sala Comercial

Os nomes de coluna acima foram abreviados só para caber na página; o que a consulta realmente devolve são `data_do_pagamento`, `valor_do_pagamento` e `codigo_imovel`, junto com a `descricao_imovel` completa de cada um (ver seção 2).

Total de registros carregados em memória: 36

ESPECIFICAÇÃO OPENAPI DOS SERVIÇOS DA PARTE 2

O enunciado pede que os três serviços da Parte 2 já sejam descritos em OpenAPI nesta entrega, mesmo antes de serem implementados. Ficaram definidos assim:

- `GET /api/imoveis/total-pagamentos` – soma de todos os pagamentos de cada imóvel (gráfico de barras);
- `GET /api/vendas/mensal` – total vendido por mês/ano (gráfico de linha ou dispersão);
- `GET /api/imoveis/percentual-por-tipo` – percentual do valor total vendido por tipo de imóvel (gráfico de pizza).

`openapi: 3.0.3`

`info:`

`title: API de Paineis Graficos de Gestao Imobiliaria - Hands on Work VII`

```
version: 1.0.0
license:
  name: MIT
servers:
  - url: http://localhost:3000
security: []

paths:
  /api/imoveis/total-pagamentos:
    get:
      summary: Total acumulado de pagamentos por imovel
      operationId: getTotalPagamentosPorImovel
      responses:
        '200':
          description: Lista com o total acumulado de pagamentos por imovel
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/TotalPorImovel'
              example:
                - codigoImovel: 3
                  totalPagamentos: 30000.00

  /api/vendas/mensal:
    get:
      summary: Total de vendas por mes/ano
      operationId: getVendasPorMes
      responses:
        '200':
          description: Lista com o total de vendas por mes/ano
```

```

content:
  application/json:
    schema:
      type: array
      items:
        $ref: '#/components/schemas/TotalPorMes'
    example:
      - mesAno: '11/2025'
        totalVendas: 23000.00

```

/api/imoveis/percentual-por-tipo:

```

get:
  summary: Percentual do valor total de vendas por tipo de imovel
  operationId: getPercentualVendasPorTipoImovel
  responses:
    '200':
      description: Lista com o percentual de vendas por tipo de imovel
      content:
        application/json:
          schema:
            type: array
            items:
              $ref: '#/components/schemas/PercentualPorTipo'
          example:
            - tipoImovel: Apartamento
              percentual: 50.0

```

components:

schemas:

TotalPorImovel:

type: object

properties:

```

    codigoImovel: { type: integer, example: 3 }
    totalPagamentos: { type: number, format: double, example: 30000.00 }
  TotalPorMes:
    type: object
    properties:
      mesAno: { type: string, example: '11/2025' }
      totalVendas: { type: number, format: double, example: 23000.00 }
  PercentualPorTipo:
    type: object
    properties:
      tipoImovel: { type: string, example: Apartamento }
      percentual: { type: number, format: double, example: 50.0 }

```

O arquivo completo está em docs/openapi.yaml, com as respostas de erro e demais detalhes que foram omitidos aqui por espaço. Ele passa sem erros pela validação do redocly lint e pode ser importado no Swagger Editor para conferência visual.

CONSIDERAÇÕES FINAIS

Com o banco modelado, a consulta com junção pronta e o código de acesso funcionando, a base para a Parte 2 já está montada: basta implementar as três funções de agregação em memória (soma por imóvel, total por mês e percentual por tipo) e expô-las como rotas REST usando Express, conforme já descrito na especificação OpenAPI.

O código-fonte completo deste trabalho está disponível no repositório do grupo, na pasta hands-on-work-vii.

REFERÊNCIAS

MICROSOFT. **TypeScript documentation**. [s. l.], 2026. Disponível em: <https://www.typescriptlang.org/docs/>. Acesso em: 3 set. 2026.

OPENAPI INITIATIVE. **OpenAPI Specification**. Version 3.0.3. [s. l.], 2021. Disponível em: <https://spec.openapis.org/oas/v3.0.3>. Acesso em: 3 set. 2026.

OPENJS FOUNDATION. **Node.js documentation**. [s. l.], 2026. Disponível em: <https://nodejs.org/docs/latest/api/>. Acesso em: 3 set. 2026.

ORACLE CORPORATION. **MySQL 8.4 Reference Manual**. [s. l.], 2024. Disponível em: <https://dev.mysql.com/doc/refman/8.4/en/>. Acesso em: 3 set. 2026.

SIDOROV, Andrey. **mysql2**. [s. l.], 2024. Disponível em: <https://www.npmjs.com/package/mysql2>. Acesso em: 3 set. 2026.

SMARTBEAR SOFTWARE. **Swagger Editor**. [s. l.], 2026. Disponível em: <https://editor.swagger.io/>. Acesso em: 3 set. 2026.

UNIVALI. **Hands on Work VII**: enunciado da atividade avaliativa. Itajaí: Universidade do Vale do Itajaí, 2026.